RESEARCH-ARTICLE

Hardware Implementation of Image Processing Morphological and Convolution Operations as SoC on FPGAs

ALFRED M. MOUSSA, Boise State University, Boise, ID, United States

RICHARD GROVES, Boise State University, Boise, ID, United States

NADER RAFLA, Boise State University, Boise, ID, United States

Published: 22 January 2025

DMIP '24: 2024 7th International Conference on Digital Medicine and Image Processing
November 8 - 11, 2024
Osaka, Japan

DMIP '24: Proceedings of the 2024 7th International Conference on Digital Medicine and Image Processing (November 2024)
https://doi.org/10.1145/3705927.3705940
ISBN: 9798400709586

Hardware Implementation of Image Processing Morphological and Convolution Operations as SoC on FPGAs

Alfred M. Moussa*
Electrical and Computer Engineering Department
Boise State University
Boise, Idaho, USA
Alfredmoussa@u.boisestate.edu

Richard Groves
Electrical and Computer Engineering Department
Boise State University
Boise, Idaho, USA
Richardgroves197@u.boisestate.edu

Nader Rafla
Electrical and Computer Engineering Department
Boise State University
Boise, Idaho, USA
Nrafla@boisestate.edu

Abstract

Efficient image processing architectures are consistently in demand across a multitude of applications, particularly those customized for resource-constrained systems-on-chip (SoC). The increasing need for high-performance image processing in various sectors has driven the development of specialized architectures. However, deploying such architectures on platforms with limited resources, such as SoCs, poses significant challenges. Furthermore, the implementation of complex algorithms to handle large datasets using software solutions often leads to slower response times, prompting exploration into hardware implementations. Field-Programmable Gate Arrays (FPGAs) are becoming popular for hardware implementations because of their attributes: low latency, connectivity, parallel computing capabilities, and flexibility. Consequently, the utilization of FPGA-based implementations has resulted in faster and more efficient performance of unique architectures tailored to specific requirements. This paper presents a novel hardware/software co-design approach to implement erosion, dilation, and neighborhood image processing operations on the FPGA development board, "Zedboard". In this approach, the FPGA is programmed by connecting it to a PC via USB, facilitating the transfer of an image pixel by pixel. The pixels are temporarily stored in on-chip DDR and accessed through DMA (Direct Memory Access) until they are requested by an interrupt signal from the Image Processing IP, at which point they are moved to line buffers for faster processing. Once processed, the image is transmitted back to the PC via UART, facilitating pixel-by-pixel transfer for verification, where it is compared with a reference image generated using Python. This comparison confirms a **99.22%** match between the processed image and the reference image, with the discrepancy occurring at the image's edges due to initial padding. Additionally, the time required to process the entire image was measured and displayed on an OLED for timing verification. which processing time amounted to **2.652 ms** which is faster than other related hardware implementation.

DMIP 2024, Osaka, Japan

ACM ISBN 979-8-4007-0958-6/24/11
https://doi.org/10.1145/3705927.3705940

CCS Concepts

• **Hardware → Board- and system-level test**; *Application-specific VLSI designs*; • **Computer systems organization → Real-time operating systems**; **Multiple instruction, single data**; • **Theory of computation → Programming logic**; **Constraint and logic programming**.

Keywords

Field Programmable Gate Array (FPGA), HW-SW co-design, System On Chip (SOC), Morphological and Convolution Operations, Hardware Implementation

ACM Reference Format:
Alfred M. Moussa, Richard Groves, and Nader Rafla. 2024. Hardware Implementation of Image Processing Morphological and Convolution Operations as SoC on FPGAs. In *2024 7th International Conference on Digital Medicine and Image Processing (DMIP) (DMIP 2024), November 08–11, 2024, Osaka, Japan.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3705927.3705940

1 Introduction

In the realm of image processing, there is a continuous pursuit for efficient architectures customized for resource-constrained systems-on-chip (SoCs), spanning various applications. The escalating demand for high-performance image processing has driven the development of specialized architectures. However, deploying these architectures on platforms with limited resources, like SoCs, presents significant challenges. Additionally, employing sophisticated algorithms for processing large datasets through software solutions often encounters delays in response time. This has led to a shift towards hardware implementations. Digital image and video processing play a crucial role across a broad spectrum of applications, from military and commercial drones to computer vision systems, surveillance and security, access control, industrial inspection and more. While some applications may not require extensive data processing, others are bound by strict requirements such as power efficiency, real-time processing and cost-effectiveness. The complexity of these demanding algorithms often makes implementation on traditional general-purpose processors impractical, making hardware implementation the preferred choice [14]. The rising performance of FPGAs can be attributed to their unique attributes: low latency, connectivity, parallel computing capabilities, and flexibility, which results in faster and more efficient outcomes [19], [21], and [23]. Consequently, this has facilitated the development of unique architectures on FPGA platforms, making hardware implementation an appealing alternative for numerous embedded and complex applications [18].Hardware implementation of image processing

algorithms on FPGAs present notable advantages over software-based methods. Such implementations enable parallel processing and pipelining, thereby expediting image processing and analysis, particularly beneficial in real-time scenarios. Today's CMOS imaging sensors typically possess resolutions of several megapixels, demanding substantial increase in data rates for processing. To overcome this obstacle, optimized memory architectures for real-time digital hardware realization for morphological and convolution operations prove to be beneficial. Moreover, FPGA-based implementation offer the flexibility to customize and optimize hardware architecture to precisely match specific processing needs, thereby enhancing overall efficiency. Furthermore, FPGA technology boasts potential benefits including lower power consumption, heightened reliability, and reduced latency when compared to conventional software-based approaches. Consequently, leveraging FPGA-based architectures holds the potential to significantly improve the performance and efficiency of image processing applications, establishing them as indispensable tools across various sectors such as medical imaging, surveillance, and autonomous vehicles.

Recognizing the critical role of digital image processing and its substantial implementation on reconfigurable FPGA devices to meet the demands of embedded systems, this study focuses on the development and execution of image processing morphological and convolution operations on FPGA. To achieve this, the Xilinx Vivado Design Suite is utilized for the development, synthesis, and implementation of the FPGA design. The aim is to demonstrate the viability of designing essential embedded applications on the FPGA platform utilizing the combination of the Programmable Logic (PL) and the Processing System (PS) of the ZYNQ-7000. This approach is suitable for image processing applications that require real time morphological and Convolution operations with minimal resource utilization for resource-constrained systems-on-chip (SoCs).

This paper's structure is as follows: Section II presents a literature review. Section III discusses the proposed methodologies and the implementation of the image processing architecture, which is further detailed in Section IV. Lastly, Section V encapsulates the findings and conclusions of this study.

2 Literature Review

In the realm of hardware-accelerated systems for image processing, particularly in the context of morphological and convolution-based solutions, various aspects come into focus. One such aspect involves the comparison and evaluation of the efficiency and performance of different solutions and algorithms tailored to specific applications. Despite the existence of numerous software-based algorithms, the demand for faster performance remains paramount, particularly in real-time applications.

Typical image and video processing tasks, such as image enhancement, object segmentation, or image measurement, involve neighborhood operations [9]. These tasks often employ a two-dimensional (2D) filter kernel or structuring element (used for shape-based morphological operations), which is shifted pixel by pixel across the entire image. During each shift step, the weighted image pixels covered by the filter kernel/structuring element undergo linear or non-linear processing operations.

2D separable operations can be broken down into a sequence of two 1D filter steps. However, this decomposition requires intermediate storage of the entire image data, which may be impractical for embedded real-time implementations due to the high memory requirements. This constraint limits parallel processing capabilities by multiport storage structures.

In general, a sequential decomposition of 2D nonlinear image processing operations, such as morphological operations, may not be feasible for all types of structuring element shapes [10].

Dilation and erosion operations form the foundation of morphological processing. Dilation enlarges components in images and connects small gaps, while erosion shrinks components in images and eliminates protrusions. Additionally, opening and closing operations are crucial in morphological processing. Opening involves erosion followed by dilation, which helps smooth contours and eliminate thin protrusions. Conversely, closing entails dilation followed by erosion, filling contour gaps and eliminating small holes [9]. Therefore, based on the functions of these operations, opening is used to extract the background, while both opening and closing operations are employed to smooth objects and eliminate small noise points in this paper.

Dilation and erosion are the foundational operators upon which all morphological operations rely. More complex operators like opening, closing, and hit-or-miss transform are built upon these fundamental operators. These fundamental operators are utilized in texture analysis and image denoising applications. Over the last decade, various architectures have been developed to enhance computational efficiency in image processing and computer vision tasks.

These architectures can be categorized into four types:

(1) Partial results reuse [3], [16], and [20]
(2) Delay line architecture [4], [8], and [12]
(3) Systolic array architecture [6], and[22]
(4) Pipelined architecture [5], [2]
(5) Cyclic image line storage [13], and [11]

These advancements aim to optimize performance and streamline computation in morphological operations and related tasks.

The first known architecture is called Partial Results Reuse (PRR) [3], initially introduced by Chien, Ma and Chen. for flat structuring elements and binary images, involves generating partial results in one operation and reusing these results in subsequent manipulations. This approach reduces redundancy by eliminating unnecessary calculations. Additionally, the system supports only flat structuring elements. To enhance efficiency, the feedback loop path in this architecture leverages partial results to reduce the use of adders and subtracters, thereby mitigating the computational and propagation delays associated with mathematical morphological operations. Furthermore, an encoder or decoder pair comparator utilizes a converted decoding function as part of this process.

Gibson, Ahmadinia, McMeekin, Strang, and Morison [8] introduced the delay line architecture, which utilizes shift registers to store pixels for current processing and line buffers to retain remaining pixels for subsequent operations. This architecture enables parallel processing where pixels are accessed from shift registers for

simultaneous manipulations. In the delay line architecture, pixels are read in raster scan mode, and line buffers are utilized for rotational accumulation of pixels. The size of delay elements and line buffers depends on the size of the structuring element. As the size of the structuring element increases, both delay elements and line buffers are scaled accordingly. This system is designed to optimize efficiency and performance in image processing tasks.

The systolic array architecture, introduced by Diamantaras and Kung [6] , is widely employed in applications such as video and image processing to enhance architectural performance. This design is specifically tailored for gray-scale erosion and dilation using flat top structuring elements. To ensure synchronization between image data and structuring elements (SEs), additional delay elements are incorporated; however, the inclusion of these delay elements can reduce the system's overall performance. In this architecture, non-flat top structuring elements can be utilized for larger sizes of structuring element used in higher resolution images. However, the performance of the system may decrease when accessing external memory. Deforges, Normand and Babel [5] introduced the pipelined architecture to support arbitrary convex structuring elements (SEs). To manage the computational complexity associated with large SE sizes and the number of dilation operations, a deep pipeline is employed. This design aims to reduce processing time by minimizing the number of comparisons conducted in parallel, thereby improving both latency and throughput.

[13] introduced a cyclic image line storage architecture (CLIS) designed for neighborhood morphological operations. This architecture operates by reading pixels in raster scan mode and cyclically storing them in a line buffer. While the design proposed by [1] boasts reconfigurability in terms of the shape and sizes of Structuring Elements (SEs). It shares a common drawback with delay-line-based architectures, namely, high memory demands and latency issues, particularly noticeable when processing images with high resolutions or larger SE sizes.

Holzer, Schumacher, Flores, Greiner and Rosenstiel [11] introduced a proficient cyclic image line storage mechanism utilizing dual-port RAM buffers. However, their implementation was focused solely on grayscale images. While the design demonstrates notable performance for a 9x9 structuring element, it might encounter challenges when dealing with larger structuring elements or higher-resolution images. This constraint could diminish the versatility and adaptability of the design in situations demanding larger structuring elements or higher-resolution image processing capabilities.

Perera and Premasir [17] suggested a straightforward yet efficient method for implementing crucial pre-processing and morphological operations of image processing on FPGA. However, their approach results in high latency due to the extensive resource utilization, compounded by image resizing performed directly on the FPGA. This latency issue impedes the feasibility of real-time applications.

Mukherjee, Mukhopadhya and Biswas [15] introduced a parallel algorithm for executing 2-D grayscale morphological operations like dilation and erosion. In this approach, the authors utilized rectangular structural elements, valued for their simplicity and straightforward implementation. However, a drawback of this architecture is its limited flexibility in handling various shapes of structuring elements during processing.

Understanding mathematical morphological theory can pose challenges, particularly regarding computational intensity and selecting appropriate shapes and scales of structuring elements (SE). This complexity is compounded by significant memory requirements, especially when dealing with high image resolutions and reconfigurable SE sizes and shapes. Consequently, Elloumi, Sellami and Rabah. [7] introduced a pipelined architecture consisting of opening and/or closing blocks (PUOC) to address these issues. However, this study stores intermediate data in registers, increasing resource consumption with SE size. Moreover, it processes array pixels simultaneously, resulting in increased latency as the image size grows.

3 IMAGE PROCESSING ON ZYNQ SOC

The architecture of the proposed hardware implementation for image processing on the Zynq System on Chip (SoC) is organized into four main modules, beyond the encompassing top module. These include: Image Control module, Line Buffer module, and Output Buffer. Each module is integral to facilitating the implementation of neighborhood processing, erosion, and dilation operations efficiently.

3.1 Image Control Module

The Image Control module, depicted in figure 1, serves as the nucleus of our image processing framework. It is responsible for managing incoming image pixels, with an 8-bit depth, directing them towards one of four Line Buffer modules. Each buffer is allocated to store a single image row, accommodating our system's specification of 512 pixels per row. As each buffer reaches capacity, control seamlessly transitions to the next buffer in sequence, cycling back after the fourth buffer is filled. This process repeats continuously until the image transmission is complete. With three rows of pixel data accumulated, the module commences the compilation of 3x3 pixel windows from these rows, proceeding one pixel at a time across each. These windows are subsequently dispatched for processing, either through morphological methods or convolution, based on the specified processing criteria.

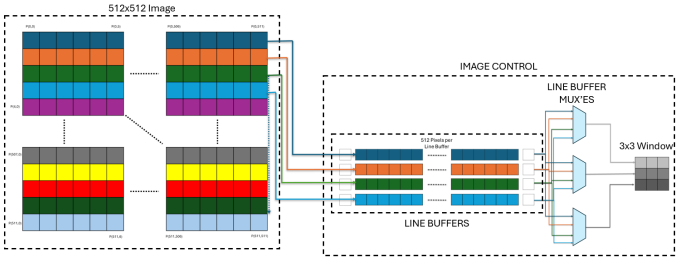

Figure 1: The Image Control Module

3.2 Line Buffer Module

The Line Buffer Module is crafted to manage the storage and sequential readout of pixel data for one-dimensional image processing. It holds up to 512 pixels, corresponding to one row of an image, and outputs data as triplets of pixels in a FIFO (First-In-First-Out) manner, crucial for creating 3x3 pixel windows. A vital function of the Line Buffer Module is to add padding to the image data as it is processed. Padding is necessary when the 3x3 window extends beyond the boundary of the image. In our design, padding is implemented at the hardware level using the following logic, where **rdPntr** represents the read pointer pointing to the pixel position in the Line Buffer:

- When the read pointer (`rdPntr`) is at the initial position (0), the module outputs a triplet where the first pixel value is assumed to be 0 (zero-padding), followed by the first and second pixels in the line buffer.
- If the read pointer (`rdPntr`) is at the end of the line (position 511), the output is a triplet consisting of the penultimate and last pixels in the line buffer, followed by a padded zero value.
- For all other cases, to ensure we stay within bounds of the Line Buffer, the output is a triplet composed of the pixel at position `rdPntr-1`, the pixel at `rdPntr`, and the pixel at `rdPntr+1`.

This approach to padding ensures that the edges of the image are properly managed during convolution without affecting the actual image data. It allows the convolution or morphological process to treat the edges of the image as if they were surrounded by zeros, a common practice in image processing to handle boundary conditions.

3.3 Convolution Module

The Convolution module (shown in Fig. 2), a pivotal component of our image processing system, is specifically tailored for convolution operations. Accepting a 3x3 window of pixels as input, this module applies a predetermined kernel to process and output a single processed pixel value.

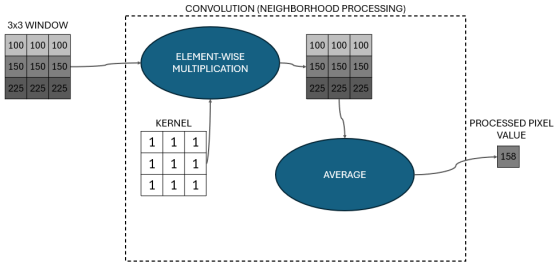

Figure 2: Convolution Module

3.4 Morphological Processing

Morphological operations within our system are executed using SystemVerilog functions for dilation and erosion, bypassing the need for separate modules. This decision leverages the combinational nature of these operations, which do not require clocking, enhancing efficiency. Both the dilate and erode functions take a 3x3 structuring element (9 bits) and a 3x3 window of pixels (72 bits for our 8-bit system), returning a singular, processed pixel value. This approach allows for streamlined integration within the top module, facilitating morphological processing with minimal overhead.

3.5 OLED Control Module

The OLED control module is specifically designed to manage the display output on the UG-2832HSWEG04 OLED screen by Univision Technology Inc., which is integrated on the Zedboard. This particular module is not a part of the image processing algorithms but rather is used for verification. It orchestrates direct communication with the OLED display through a Serial Peripheral Interface (SPI), facilitating the transmission of graphical data and control commands. A key feature of this module is its interaction with the Timer module; the value acquired from the Timer, indicative of the processing time for each image by recording the total number of clock cycles required to process the entire image, from the first pixel to the last. This processing time value is prominently displayed on the OLED screen. This feedback mechanism allows for the immediate visualization of the system's performance, enhancing user interaction by providing a tangible measure of the image processing efficiency. The display of the timer's value not only serves as a diagnostic tool but also enables performance benchmarking and system optimization by illustrating the processing times directly on the OLED. In Figure 3 illustrates the designed controller Finite Sate Machine for the OLED.

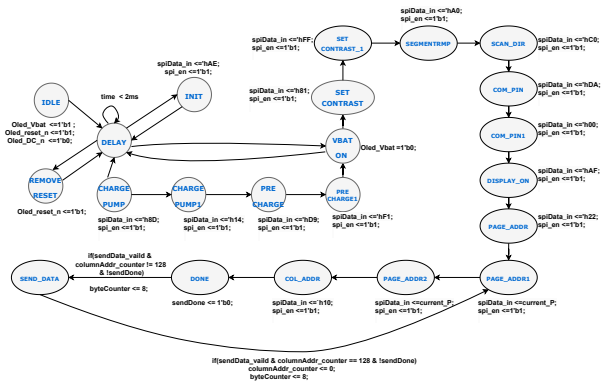

Figure 3: OLED Finite State Machine (FSM)

3.6 Timer Module

The Timer module is essential for capturing the total number of clock cycles needed to process an entire image, providing critical performance metrics for analysis and optimization of image operations. Integrated into the FPGA's logic, this module generates precise timing signals aligned with the system clock, achieving microsecond-level accuracy in timing measurements. By measuring the total image processing time, the Timer module enables optimizations to meet real-time processing requirements effectively. The processing time for a 512x512 image as 0x40a00, equivalent to 264,704 in decimal and approximately 2.6470 milliseconds.

3.7 Output Buffer

To address the disparity between the processing speed of the image data and the slower transmission capabilities of UART, the Output Buffer was implemented using the Xilinx FIFO Generator IP. This module serves as a temporary holding area for processed pixels, ensuring that data is transmitted over UART to the PC without loss. It is capable of accepting one pixel at a time from the top module, mitigating any bottlenecks in data flow from processing to transmission.

3.8 Testbench Module

To evaluate the functional behavior and a timing of the design architecture, simulation was conducted as preliminary step before real implementation on the ZedBoard. These simulations entailed reading a 512x512 grayscale bitmap image from the PC and sequentially transmitting pixels to the top module, replicating the behavior post-implementation. Additionally, the testbench module received processed pixels from the top module, aggregating them into a new bitmap image file. This resultant image was subsequently compared with a referenced image generated by Python to validate the processing outcome pixels, demonstrating a processing time of 2.647 ms for the intended functional.

4 Full System Architecture

The proposed Image Processing system was meticulously designed and implemented using Verilog and System Verilog hardware description languages, specifically tailored for the Zedboard FPGA. A block diagram of the system architecture, is shown in figure 4, is packaged as a robust hardware IP module. This system seamlessly integrates with the on-chip ARM processor via the AXI4 interconnect interface, to efficiently managing data translation between the AXI bus and the necessary input/output signals for the Image Processing IP. Additionally, the AXI Direct Memory Access (AXI DMA) is leveraged to handle high-bandwidth access between memory and AXI4-Stream-compatible peripherals. The image was stored in external DDR memory and the DMA was configured to stream image data from the memory to the Image Processing IP. The Image Controller reads the data through the AXI4 interface and forwards it to the IP using the AXI4 Stream interface. The Image Processing IP processes the image, handling multiple pixels simultaneously until the four line buffers are filled. Once a line buffer is fully processed, the IP sends an interrupt signal to the DMA to request filling the empty line buffer and streams back the processed data to the DMA Controller. This setup ensures suitability for real-time applications. The DMA Controller receives the stream data from the IP and transfers it to external DDR memory via the AXI4 interface, subsequently transmitting it over UART. This setup optimizes execution time and efficiency in managing data transmission from DDR to the Image Processing IP, ensuring smooth communication and data flow within the system's architecture. Designed to operate at a clock frequency of 100MHz, the system is finely tuned for real-time processing requirements, demonstrating its capacity for high-performance image analysis.

An evaluation of the FPGA resources employed for the system's implementation was facilitated by the Vivado design suite. This

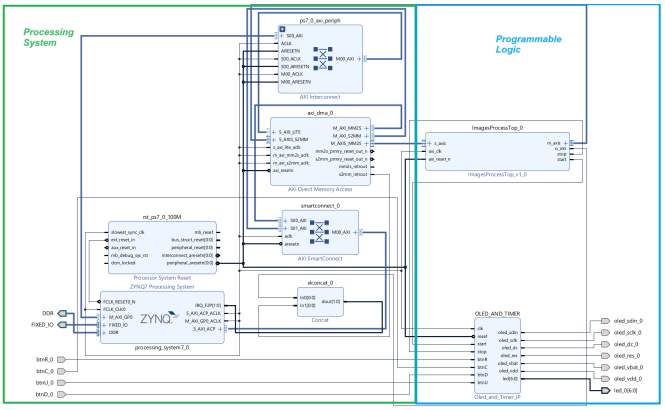

Figure 4: System Block diagram illustrating the integration of Image Processing IP with the Processing System (ARM Microcontroller).

analysis not only delineates the efficient allocation of FPGA resources, including slices, Look-Up Tables (LUTs), flip-flops, Block RAMs (BRAMs), and Digital Signal Processors (DSPs), but also underscores the system's energy efficiency through detailed power consumption metrics. Such an assessment highlights the design's ability to harness FPGA capabilities for advanced image processing tasks, ensuring minimal power expenditure while delivering superior performance.

5 Conclusion

This paper presents a real-time image processing module that has been developed and implemented on an FPGA. The module can be configured to process any number of 512 x 512 images and send the processed images back to a PC via a UART interface. The design utilizes BRAM blocks to store the results of arithmetic operations. Using on-chip BRAM is preferred over RAM for their low latency. The design uses fewer hardware resources and consumed less power when compared to latest architectures for Digital Image Processing at the same clock frequency and higher. The transformations applied on the original Lena and Cameraman images, as shown in figure 5, demonstrate the module's high accuracy and reliability in real-time processing.

(a) (b) (c) (d)

Figure 5: Comparison of original images (a) Lena and (c) Cameraman with their respective processed outputs (b) convolution and (d) erosion.

Table 1 presents a comparative analysis with latest Digital Image Processing architectures implemented on FPGA.

Table 1: COMPARISON BETWEEN OTHER IMPLEMENTATION .

Mode		Our		[1]	[15]	[24]
FPGA Platform		Zedboard		Zedboard	VIRTEX-5	EP2S180
Image Size		512X512		-	256x256	1024X1024
Img.Proc	neighborhood	Dilation	erosion	Dilation & erosion	Dilation & erosion	Dilation & erosion
Clock (MHZ)	100	100	100	100	-	220
LUTRAM	1154	1153	1153	252	-	
LUT	1701	2170	2158	4778	1901	
DSP48E	0	0	0	9	-	
BRAM	0.5	0.5	0.5	4.5	-	
BUFG	1	1	1	1	-	
FF	270	200	200	5081	1193	
Processing Time	2.6470 ms	2.6470 ms	2.6470 ms	7.337 ms	-	4.78 ms

For instance, in [1], a ZedBoard (the same board used in this research) was utilized, focusing solely on reporting resource utilization and processing time without specifying the image size, which allows for a meaningful comparison. This observation also applies to authors [15], and [24]. In contrast to approaches in [1] and [15], which handle one kernel or pixel at a time using a single line buffer, this study employs parallel and pipelining methods with partial reuse and cyclic image line storage for Morphological Processing. Although [1] employs only 9 DSPs, their significant power consumption and potential resource limitations are notable concerns. Furthermore, the use of the OLED and Timer module underscores the hardware-based processing time of this architecture, rather than relying on simulation tools. The utilization of BRAM for storing pixel values in line buffers is crucial and is benchmarked against the resource utilization of a 3x3 kernel structural element to highlight comparisons with previous architectures.

References

[1] M. S. Azzaz, A. Maali, R. Kaibou, I. Kakouche, M. Saad, and H. Hamil. 2020. FPGA HW/SW Codesign Approach for Real-time Image Processing Using HLS. In *2020 1st International Conference on Communications, Control Systems and Signal Processing (CCSSP)*. El Oued, Algeria, 169–174. https://doi.org/10.1109/CCSSP49278.2020.9151686

[2] J. Bartovsky, P. Dokladal, E. Dokladalova, and V. Georgiev. 2014. Parallel implementation of sequential morphological filters. *Journal of Real-Time Image Processing* 9, 2 (2014), 315–327.

[3] Shao-Yi Chien, Shyh-Yih Ma, and Liang-Gee Chen. 2005. Partial-result-reuse architecture and its design technique for morphological operations with flat structuring elements. *IEEE Transactions on Circuits and Systems for Video Technology* 15, 9 (2005), 1156–1169. https://doi.org/10.1109/TCSVT.2005.852622

[4] C. Clienti, S. Beucher, and M. Bilodeau. 2008. A system on chip dedicated to pipeline neighborhood processing for mathematical morphology. In *Signal Processing Conference, 2008 16th European*. IEEE, 1–5.

[5] O. Deforges, N. Normand, and M. Babel. 2013. Fast recursive grayscale morphology operators: from the algorithm to the pipeline architecture. *Journal of Real-Time Image Processing* 8, 2 (2013), 143–152.

[6] K. I. Diamantaras and S.-Y. Kung. 1997. A linear systolic array for real-time morphological image processing. *Journal of VLSI Signal Processing Systems for Signal, Image and Video Technology* 17, 1 (1997), 43–55.

[7] H. Elloumi, D. Sellami, and H. Rabah. 2022. A Flexible Hardware Accelerator for Morphological Filters on FPGA. In *2022 8th International Conference on Control, Decision and Information Technologies (CoDIT)*. Istanbul, Turkey, 550–555. https://doi.org/10.1109/CoDIT55151.2022.9804025

[8] R. M. Gibson, A. Ahmadinia, S. G. McMeekin, N. C. Strang, and G. Morison. 2013. A reconfigurable real-time morphological system for augmented vision. *EURASIP Journal on Advances in Signal Processing* 2013, 1 (2013), 1–13.

[9] WOODS R. E GONZALEZ, R. C. 2008. *Digital Image Processing* (3rd ed.).

[10] Ronaldo Fumio Hashimoto, Junior Barrera, and Carlos Eduardo Ferreira. 2000. A Combinatorial Optimization Technique for the Sequential Decomposition of Erosions and Dilations. *Journal of Mathematical Imaging and Vision* 13, 1 (Aug. 2000), 17–33. https://doi.org/10.1023/A:1008373522375

[11] M. Holzer, F. Schumacher, I. Flores, T. Greiner, and W. Rosenstiel. 2011. A real time video processing framework for hardware realization of neighborhood operations with FPGAs. In *21st International Conference Radioelektronika 2011*. Brno, Czech Republic, 1–4. https://doi.org/10.1109/RADIOELEK.2011.5936393

[12] M. Holzer, F. Schumacher, T. Greiner, and W. Rosenstiel. 2012. Optimized hardware architecture of a smart camera with novel cyclic image line storage structures for morphological raster scan image processing. In *Emerging Signal Processing Applications (ESPA), 2012 IEEE International Conference on*. IEEE, 83–86.

[13] M. Holzer, F. Schumacher, T. Greiner, and W. Rosenstiel. 2012. Optimized hardware architecture of a smart camera with novel cyclic image line storage structures for morphological raster scan image processing. In *Emerging Signal Processing Applications (ESPA), 2012 IEEE International Conference on*. IEEE, 83–86.

[14] Jaber Hosseinzadeh and Maghsoud Hosseinzadeh. 2016. A Comprehensive Survey on Evaluation of Lightweight Symmetric Ciphers: Hardware and Software Implementation. *Advances in Computer Science* 5 (07 2016). https://www.acsij.org/index.php/acsij/article/view/529

[15] D. Mukherjee, S. Mukhopadhyay, and G. P. Biswas. 2016. FPGA based parallel implementation of morphological filters. In *2016 International Conference on Microelectronics, Computing and Communications*.

[16] Soohwan Ong and M.H. Sunwoo. 2000. A morphological filter chip using a modified decoding function. *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* 47, 9 (2000), 876–885. https://doi.org/10.1109/82.868455

[17] R. Perera and S. Premasiri. 2017. Hardware implementation of essential preprocessing and Morphological Operations in Image Processing. In *2017 6th National Conference on Technology and Management (NCTM)*. Malabe, Sri Lanka, 142–146. https://doi.org/10.1109/NCTM.2017.7872843

[18] Neeraj Shukla B. K. Das Prachi Dewan, Rekha Vig. 2016. Novel VLSI Architectures for Image Segmentation and Edge Detection Algorithm. *International Journal of Computer Applications* 149, 10 (Sep 2016), 32–36. https://doi.org/10.5120/ijca2016911577

[19] Oliver Reiche, Mehmet Akif Özkan, Frank Hannig, Jürgen Teich, and Moritz Schmid. 2018. Loop Parallelization Techniques for FPGA Accelerator Synthesis. *Journal of Signal Processing Systems* 90 (2018), 3–27.

[20] M.-H. Sheu, J.-F. Wang, J.-S. Chen, A.-N. Suen, Y.-L. Jeang, and J.-Y. Lee. 1992. A data-reuse architecture for gray-scale morphologic operations. *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* 39, 10 (1992), 753–756. https://doi.org/10.1109/82.199903

[21] Fahad Siddiqui, Sam Amiri, Umar Ibrahim Minhas, Tiantai Deng, Roger Woods, Karen Rafferty, and Daniel Crookes. 2019. FPGA-Based Processor Acceleration for Image Processing Applications. *Journal of Imaging* 5, 1 (2019). https://doi.org/10.3390/jimaging5010016

[22] C. Torres-Huitzil. 2013. Fast hardware architecture for grey level image morphology with flat structuring elements. *IET Image Processing* 8, 2 (2013), 112–121.

[23] Steven Vlugt, Hadi Alizadeh Ara, Rob Jong, Martijn Hendriks, Ruben Guerra Marin, Marc Geilen, and Dip Goswami. 2019. Modeling and Analysis of FPGA Accelerators for Real-Time Streaming Video Processing in the Healthcare Domain. *J. Signal Process. Syst.* 91, 1 (Jan. 2019), 75–91. https://doi.org/10.1007/s11265-018-1414-3

[24] B. Zhang, K. Mei, and N. Zheng. 2013. Reconfigurable Processor for Binary Image Processing. *IEEE Transactions on Circuits and Systems for Video Technology* 23, 5 (May 2013), 823–831. https://doi.org/10.1109/TCSVT.2012.2223872